

SOLID Architecture Cheat Sheet

Clean Object-Oriented Design Applied to Spring Boot Microservices

REVISION NOTES

S Single Responsibility Principle (SRP)

Rule: One Class = One Single Responsibility (Only one reason to change).

✗ Violation: Multiple Reasons to Change

```
@Service
public class UserService {
    public User getUser(Long id) {
        return userRepo.findById(id).get();
    }
    public void sendEmail(User u) { /* email */ }
    public void generatePdf(User u) { /* pdf */ }
}
```

Risk: Email provider updates or PDF layout changes force modifications to UserService.

✓ Refactored: Focused Micro-Services

```
@Service
public class UserService {
    public User getUser(Long id) {
        return userRepo.findById(id).get();
    }
}
@Service public class EmailService { ... }
@Service public class PdfService { ... }
```

Result: Each class has exactly one bounded context and one driver of change.

🌱 How Spring Boot Enforces SRP:

- **@RestController:** Exclusively handles HTTP routing, request parsing, and HTTP responses.
- **@Service:** Contains deterministic domain business logic and transactions.
- **@Repository:** Exclusively manages persistent data access and ORM entity mapping.

O Open/Closed Principle (OCP)

Rule: Open for Extension, Closed for Modification (Add features by writing new code, not editing existing code).

✗ Violation: Chained Conditional Logic

```
public class PaymentService {
    public void pay(String type) {
        if (type.equals("UPI")) { /* ... */ }
        else if (type.equals("CARD")) { /* ... */ }
        else if (type.equals("PAYTM")) { /* ... */ }
    }
}
```

Risk: Introducing ApplePay requires editing and re-testing this critical shared class.

✓ Strategy Pattern via Spring Beans

```
public interface PaymentStrategy { void pay(); }

@Component
public class UpiPayment implements PaymentStrategy { ... }

@Component
public class PaytmPayment implements PaymentStrategy { ... }
```

Result: Simply drop in a new bean implementing the interface. Zero editing of existing code.

🌱 How Spring Boot Implements OCP:

Spring's Dependency Injection container automatically scans and injects new implementations (e.g. `List<PaymentStrategy>`) at runtime without requiring framework modification.

L Liskov Substitution Principle (LSP)

Rule: Subtypes must be substitutable for their base types without breaking application correctness.

✗ Violation: Throwing Unexpected Exceptions

```
interface Bird { void fly(); }

class Penguin implements Bird {
    public void fly() {
        // Violates caller expectations!
        throw new UnsupportedOperationException();
    }
}
```

Risk: A client looping over a collection of `Bird` crashes when invoking `fly()`.

✓ Segregated Subtypes & Capabilities

```
interface Bird { /* shared bird traits */ }
interface FlyingBird extends Bird {
    void fly();
}

class Sparrow implements FlyingBird { ... }
class Penguin implements Bird { ... }
```

Result: Subtypes fulfill 100% of their inherited contracts with no runtime surprises.

🌱 How Spring Boot Implements LSP:

Spring Data's `JpaRepository` interface allows swapping backing implementations (PostgreSQL, MySQL, H2 for testing) without altering consumer code. Any bean matching the contract behaves predictably for operations like `save()`, `findById()`, and `delete()`.

I Interface Segregation Principle (ISP)

Rule: Clients should not be forced to depend upon methods they do not use (Prefer small, focused interfaces).

✗ Violation: Bloating "Fat" Interface

```
public interface Worker {
    void work();
    void eat();
    void sleep();
}

class Robot implements Worker {
    public void work() { ... }
    public void eat() { /* forced empty body */ }
    public void sleep() { /* meaningless */ }
}
```

✓ Composable Single-Purpose Interfaces

```
interface Workable { void work(); }
interface Eatable { void eat(); }

class Robot implements Workable { ... }

class Human implements Workable, Eatable {
    public void work() { ... }
    public void eat() { ... }
}
```

🌱 How Spring Boot Implements ISP:

Spring framework uses atomic interfaces instead of one monolith. For example, `UserDetails` in Spring Security strictly handles authentication contracts; auditing uses `AuditorAware`; and serialization uses `Serializable`.

D Dependency Inversion Principle (DIP)

Rule: High-level modules should depend on abstractions (interfaces), never on low-level concrete implementations.

✗ Violation: Tightly Coupled Concrete Class

```
@Service
public class OrderService {
    // Direct concrete coupling!
    private MySqlRepo repo = new MySqlRepo();

    public void saveOrder() {
        repo.save();
    }
}
```

Problem: Migrating from MySQL to Mongo or writing unit tests requires editing OrderService.

✓ Loose Coupling with Dependency Injection

```
@Service
public class OrderService {
    private final OrderRepository repo;

    // Constructor-based Injection
    public OrderService(OrderRepository repo) {
        this.repo = repo;
    }
}
```

Result: High-level service is completely agnostic to whether MySQL, Mongo, or MockRepo is injected.

🌱 The Core Magic: Inversion of Control (IoC) & DI

Spring Boot's entire ecosystem is built around DIP. The IoC container creates and manages concrete instances, resolving and injecting dependencies via constructor injection at runtime.

⚡ 2-MINUTE INTERVIEW CHEAT SHEET: "HOW DOES SPRING BOOT FOLLOW SOLID?"

QUICK
ANSWER

Principle	Core Definition	Spring Boot Direct Example
S - SRP	One class, one reason to change	Layered separation: @RestController (HTTP), @Service (Business), and @Repository (DB access).
O - OCP	Open extension, closed modification	Registering new @Component or @Bean implementations without altering core framework code.
L - LSP	Subtypes preserve contract behavior	Injecting JpaRepository; switching database drivers or mocking repositories does not break callers.
I - ISP	Small, decoupled interfaces	Granular contracts like UserDetails, CommandLineRunner, or AuditorAware.
D - DIP	Depend on abstractions, not concretions	Spring's IoC & Dependency Injection; controllers and services depend on repository interfaces.